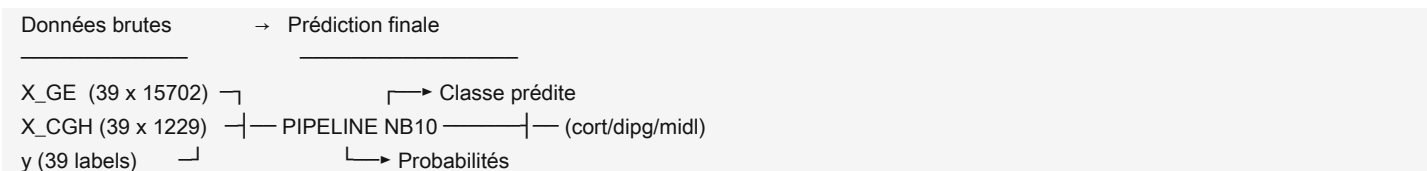


Pipeline Cooperative Learning + LDA

Description complète de l'implémentation NB10

Vue d'ensemble

Le pipeline NB10 transforme les données brutes (X_GE 39x15702, X_CGH 39x1229, y 39 labels) en une classe prédite cort/dipg/midl via 6 étapes successives.



Étape 1 — Chargement, alignement, imputation

Lecture des CSV bruts, conversion des virgules en points, alignement des patient IDs entre les 3 fichiers (intersection), puis imputation médiane par le train uniquement (anti-fuite). Conversion en matrix R pour multiview.

Étape 2 — Encodage one-vs-rest (OvR)

multiview ne supporte pas family=multinomial. On transforme le problème 3-classes en 3 problèmes binaires indépendants. La cible devient une matrice one-hot 39x3 :

y_train (factor)	y_onehot (matrice 39 x 3)
cort	cort dipg midl
dipg	→ patient_1 1 0 0
midl	patient_2 0 1 0
cort	patient_3 0 0 1
...	...

Chaque colonne définit un sous-problème : cort vs reste, dipg vs reste, midl vs reste.

Étape 3 — Cooperative learning sur chaque sous-problème

Pour chaque classe $k \in \{\text{cort}, \text{dipg}, \text{midl}\}$, on lance `cv.multiview()` sur le y binaire correspondant. C'est le cœur de cooperative learning.

Ce que multiview résout (objectif)

$$\begin{aligned} \min_{\theta_GE, \theta_CGH} & -\log L(y_k | \text{softmax}(X_GE \theta_GE + X_CGH \theta_CGH + b)) \\ & \leftarrow \text{perte logistique binaire pour cette classe } k \\ & + (\rho/2) \cdot \|X_GE \theta_GE - X_CGH \theta_CGH\|^2 \\ & \leftarrow \text{terme d'agrément (à } \rho=0 \text{ il disparaît)} \\ & + \lambda \cdot (|\theta_GE|_1 + |\theta_CGH|_1) \\ & \leftarrow \text{sparsité L1} \end{aligned}$$

À $\rho=0$ (notre régime de travail), le terme d'agrément est nul et cooperative dégénère en Lasso multi-bloc avec régression logistique binaire.

Sélection de λ

`cv.multiview` fait une 5-fold CV interne pour sélectionner `lambda.min` (deviance moyenne minimale). Typiquement 100 valeurs sur grille log-décroissante.

Ce qui sort de l'étape 3

Un vecteur de coefficients concaténé pour chaque classe k :

[intercept | β_GE (15702 valeurs) | β_CGH (1229 valeurs)]

Au total après 3 fits, on a 3 jeux de coefficients (sparses) :

Classe cort : $\beta_GE^{\text{cort}}, \beta_CGH^{\text{cort}}$
Classe dipg : $\beta_GE^{\text{dipg}}, \beta_CGH^{\text{dipg}}$

Classe midl : β_{GE}^{midl} , β_{CGH}^{midl}

Étape 4 — Calcul des scores par patient

Pour chaque patient i , on calcule 6 scores numériques :

```
score_GE^cort(i) = X_GE[i, :] · β_GE^cort
score_GE^dipg(i) = X_GE[i, :] · β_GE^dipg
score_GE^midl(i) = X_GE[i, :] · β_GE^midl
score_CGH^cort(i) = X_CGH[i, :] · β_CGH^cort
score_CGH^dipg(i) = X_CGH[i, :] · β_CGH^dipg
score_CGH^midl(i) = X_CGH[i, :] · β_CGH^midl
```

Résultat : une matrice 39x6 où chaque ligne est un patient et chaque colonne représente 'à quel point ce patient ressemble à la classe X selon le bloc Y' selon le modèle cooperative.

Le piège des colonnes constantes (corrigé)

Pour midl, $\lambda_{\min}$ peut être tellement grand que tous les β^{midl} sont à zéro. Les scores midl deviennent constants à zéro pour tous les patients, ce qui fait crasher LDA. Le fix : filtrage des colonnes constantes avant LDA + sauvegarde d'un masque pour appliquer le même filtre au test.

Étape 5 — Entraînement de LDA sur les scores

LDA suppose que dans cet espace 6D, chaque classe suit une gaussienne avec covariance partagée. Elle estime :

- 3 vecteurs moyens μ_{cort} , μ_{dipg} , μ_{midl} (dimension 6 chacun)
- 1 matrice de covariance pooled Σ (6x6)
- 3 probabilités a priori π_{cort} , π_{dipg} , π_{midl}

LDA calcule ensuite les frontières de décision linéaires qui séparent au mieux les 3 classes au sens du discriminant gaussien. LDA peut aussi être vu comme un second réducteur de dimension : il projette les 6 scores dans un espace 2D ($K-1=2$ pour 3 classes) où la séparation entre classes est maximale.

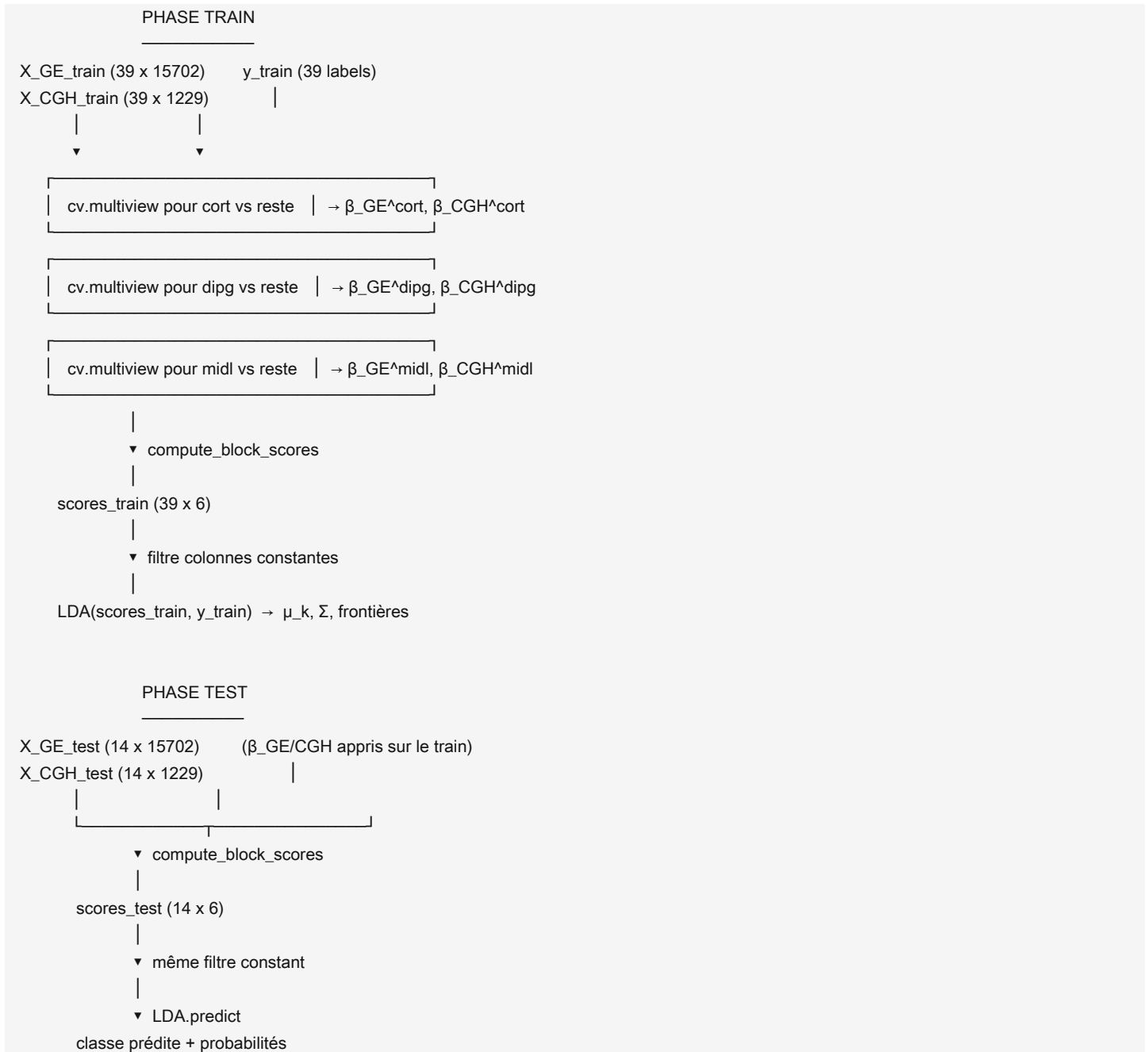
Étape 6 — Prédiction sur le test

Un nouveau patient passe par les mêmes étapes, en réutilisant les coefficients β appris sur le train et le modèle LDA déjà entraîné :

```
X_GE_test[i, :], X_CGH_test[i, :]
|
▼ compute_block_scores (réutilise les β du train)
scores_test[i, :] (6 nombres)
|
▼ filtre colonnes constantes (même masque qu'au train)
|
▼ LDA.predict (déjà entraîné)
classe prédite + posterior
```

Aucun ré-entraînement sur le test. Tout ce qui est appris sur le train est appliqué tel quel au test.

Récapitulatif global du pipeline



État actuel de la pipeline

Tableau des étapes :

Étape	État
1. Chargement + imputation	Fonctionne
2. Encodage OvR	Fonctionne
3. cv.multiview (3 fits)	Fonctionne à $p=0$ Plante à $p>0$ (non-convergence)
4. Compute block scores	Fonctionne (avec filtrage)
5. LDA	Fonctionne sur scores filtrés
6. Prédiction test	Fonctionne

Résultat à $p=0$: CV balanced accuracy = 0.829 ± 0.148 sur 21 folds. Statistiquement identique à SGCCA (NB09).

Pour $p>0$: le solveur glmnet ne converge pas à cause du mauvais conditionnement induit par le terme d'agrément. On a interrompu et fixé `best_rho = 0`.

Il reste à lancer la cellule 13 (refit final + évaluation test) puis la cellule 17 (tableau comparatif vs NB09) sur Windows.